

**BỘ KHOA HỌC VÀ CÔNG NGHỆ  
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG  
KHOA CÔNG NGHỆ THÔNG TIN 1**

-----



**BÁO CÁO BÀI TẬP LỚN  
MÔN: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG**

**ĐỀ TÀI: XÂY DỰNG WEBSITE QUẢN LÝ THU CHI CÁ NHÂN**

**Nhóm lớp 15 – Nhóm 14**

**Thành viên nhóm**

| <b>Họ tên</b>             | <b>Mã sinh viên</b> | <b>Lớp</b>         |
|---------------------------|---------------------|--------------------|
| <b>Võ Văn Bảo Phúc</b>    | <b>B21DCCN595</b>   | <b>D21HTTT04</b>   |
| <b>Trần Thị Ánh Dương</b> | <b>B23DCCN230</b>   | <b>D23CQCN06-B</b> |
| <b>Vũ Ngọc Diệp</b>       | <b>B23DCCN156</b>   | <b>D23CQCN02-B</b> |
| <b>Bùi Huy Hoàng</b>      | <b>B23DCCN324</b>   | <b>D23CQCN02-B</b> |

**Hà Nội – 2025**

## MỤC LỤC

|       |                                                  |    |
|-------|--------------------------------------------------|----|
| I.    | Giới thiệu đề tài .....                          | 4  |
| 1.    | Lý do chọn đề tài.....                           | 4  |
| 2.    | Mục tiêu hệ thống .....                          | 4  |
| 3.    | Phạm vi và đối tượng sử dụng.....                | 4  |
| 4.    | Các chức năng chính của hệ thống .....           | 5  |
| 5.    | Tổng quan các module.....                        | 5  |
| II.   | Phân tích nghiệp vụ.....                         | 6  |
| 1.    | Mô tả bài toán .....                             | 6  |
| 2.    | Quy trình nghiệp vụ chung .....                  | 6  |
| 3.    | Use Case tổng thể hệ thống .....                 | 7  |
| 4.    | Use Case theo từng module.....                   | 7  |
| III.  | Công nghệ và kiến trúc hệ thống .....            | 8  |
| 1.    | Công nghệ sử dụng .....                          | 8  |
| 2.    | Kiến trúc tổng thể hệ thống .....                | 8  |
| IV.   | Thiết kế hướng đối tượng.....                    | 9  |
| 1.    | Tổng quan 4 tính chất OOP.....                   | 9  |
| 2.    | Áp dụng OOP vào Spring Boot .....                | 10 |
| 3.    | Mô hình lớp .....                                | 11 |
| 4.    | Design Pattern áp dụng .....                     | 12 |
| V.    | Thiết kế cơ sở dữ liệu .....                     | 12 |
| 1.    | Phân tích dữ liệu .....                          | 12 |
| 2.    | ERD hệ thống .....                               | 13 |
| 3.    | Thiết kế bảng.....                               | 14 |
| 4.    | Ràng buộc và mối quan hệ dữ liệu .....           | 15 |
| 5.    | Bảng mẫu .....                                   | 15 |
| VI.   | Khai triển hệ thống .....                        | 16 |
| 1.    | Tổ chức module của hệ thống.....                 | 16 |
| 2.    | Cách các module được khởi tạo và hoạt động ..... | 18 |
| 3.    | Backend – Frontend giao tiếp.....                | 18 |
| 4.    | Một vài API mẫu.....                             | 19 |
| 5.    | Ảnh giao diện chính.....                         | 20 |
| VII.  | Hướng dẫn sử dụng hệ thống.....                  | 24 |
| 1.    | Chuẩn bị môi trường .....                        | 24 |
| 2.    | Khởi tạo cơ sở dữ liệu MySQL.....                | 24 |
| 3.    | Chạy Backend .....                               | 24 |
| 4.    | Chạy Frontend .....                              | 24 |
| 5.    | Đăng kí/ đăng nhập và sử dụng.....               | 24 |
| VIII. | Đánh giá và kết luận.....                        | 25 |
| 1.    | Ưu điểm.....                                     | 25 |

|     |                                     |    |
|-----|-------------------------------------|----|
| 2.  | Hạn chế.....                        | 25 |
| 3.  | Hướng phát triển .....              | 26 |
| IX. | Phân công và mức độ đóng góp .....  | 26 |
| 1.  | Bảng phân chia công việc .....      | 26 |
| 2.  | Tỷ lệ đóng góp từng thành viên..... | 27 |

## I. Giới thiệu đề tài

### 1. Lý do chọn đề tài

Trong bối cảnh công nghệ phát triển mạnh mẽ, nhu cầu quản lý tài chính cá nhân ngày càng trở nên quan trọng. Sinh viên hay người đi làm hay bất kỳ ai đều thường xuyên gặp những câu hỏi quen thuộc như: "Tiền đã chi vào những gì?", "Tháng này có vượt chi không?", hoặc "Làm sao quản lý nhiều ví cho gọn?".

Mặc dù đã có nhiều ứng dụng trên thị trường, phần lớn lại khá phức tạp hoặc chưa phù hợp với thói quen sử dụng của người dùng. Vì vậy, nhóm em chọn xây dựng Vissmart – trang web quản lý thu chi cá nhân với mục tiêu tạo ra một công cụ gọn nhẹ, trực quan và dễ tiếp cận nhưng vẫn đáp ứng đầy đủ các tính năng cần thiết để hỗ trợ người dùng kiểm soát tài chính hiệu quả.

Đồng thời, đây cũng là cơ hội để nhóm áp dụng kiến thức về lập trình hướng đối tượng, thiết kế hệ thống và mô hình dữ liệu vào một sản phẩm thực tế - phù hợp với tinh thần của môn học OOP.

### 2. Mục tiêu hệ thống

Hệ thống được xây dựng với các mục tiêu chính:

- Giúp người dùng quản lý chi tiêu khoa học thông qua giao dịch, ví tiền và ngân sách.
- Cung cấp góc nhìn tổng quan về tài chính cá nhân, từ đó hỗ trợ đưa ra các quyết định chi tiêu hợp lý.
- Tích hợp phân tích và gợi ý thông minh, giúp người dùng nhận diện thói quen chi tiêu và tối ưu hóa tài chính.
- Đảm bảo an toàn dữ liệu với cơ chế xác thực JWT, phân quyền và mã hóa mật khẩu.
- Tạo trải nghiệm hiện đại và thân thiện thông qua giao diện web React và API chuẩn REST từ Spring Boot.

Mục tiêu cuối cùng là mang đến một hệ thống có tính ứng dụng thật sự chứ không chỉ là bài tập lập trình.

### 3. Phạm vi và đối tượng sử dụng

#### - Phạm vi:

Hệ thống tập trung xử lý các nghiệp vụ liên quan đến tài chính cá nhân, bao gồm:

- o Quản lý tài khoản và ví
- o Ghi nhận và phân loại giao dịch
- o Theo dõi và cảnh báo ngân sách
- o Phân tích và báo cáo chi tiêu
- o Tích hợp chatbot AI để hỗ trợ người dùng

Hệ thống hoạt động trên môi trường web (React cho frontend, Spring Boot cho backend, MySQL cho cơ sở dữ liệu).

#### - Đối tượng sử dụng:

- o Sinh viên muốn kiểm soát chi tiêu hàng ngày.
- o Người đi làm có nhu cầu quản lý nhiều nguồn tài chính.
- o Những ai mong muốn theo dõi và tối ưu thói quen chi tiêu bằng công nghệ.

#### 4. Các chức năng chính của hệ thống

Dựa trên yêu cầu thực tế, hệ thống cung cấp các nhóm chức năng chính sau:

- **Xác thực & phân quyền người dùng:** đăng ký, đăng nhập, quản lý tài khoản, mã hóa mật khẩu (BCrypt), JWT.
- **Quản lý giao dịch:** thêm/sửa/xóa giao dịch, phân loại chi tiêu, tìm kiếm – lọc theo thời gian hoặc danh mục.
- **Quản lý ví & tài khoản:** theo dõi nhiều ví, cập nhật số dư theo thời gian thực, hỗ trợ các loại tài khoản khác nhau.
- **Quản lý ngân sách:** thiết lập ngân sách theo danh mục, cảnh báo vượt hạn mức.
- **Phân tích & báo cáo:** biểu đồ trực quan, báo cáo chi tiêu, phân tích xu hướng.
- **Tích hợp AI:** chatbot hỗ trợ hỏi đáp tài chính, gợi ý tối ưu chi tiêu.
- **Cài đặt & cấu hình:** tùy chỉnh thông báo, sao lưu dữ liệu, tùy chọn người dùng.
- **Trang Dashboard tổng quan:** hiển thị snapshot nhanh về tài chính trong tháng.

#### 5. Tổng quan các module

Hệ thống được chia thành các module chính, mỗi module tương ứng với một nhóm chức năng rõ ràng. Mỗi module đều có các trang giao diện cụ thể để hỗ trợ người dùng thao tác dễ dàng.

##### a) Authentication & Authorization Module

- LoginPage.tsx (/login)
- RegisterPage.tsx (/register)
- Xử lý JWT, phân quyền, quản lý thông tin người dùng

##### b) Transaction Management Module

- TransactionManagementPage.tsx (/transactions)
- Thêm/sửa/xóa giao dịch, phân loại, tìm kiếm, phân tích nhanh

##### c) Wallet & Account Management Module

- AccountManagementPage.tsx (/accounts)
- AddAccountPage.tsx (/accounts/add)
- Quản lý nhiều ví, số dư, loại tài khoản

##### d) Budget Management Module

- BudgetManagementPage.tsx (/budgets)
- Tạo ngân sách, theo dõi, cảnh báo

##### e) Analytics & Reporting Module

- ExpenseAnalysisPage.tsx (/analytics)
- ReportsPage.tsx (/reports)
- Biểu đồ, báo cáo chi tiêu, phân tích xu hướng

##### f) AI Integration Module

- ChatPage.tsx (/chat)
- Chatbot AI, gợi ý, phân tích dự đoán

##### g) Dashboard & Overview Module

- DashboardPage.tsx (/dashboard)
- Tổng quan tài chính theo thời gian thực

##### h) Configuration & Settings Module

- SettingsPage.tsx (/settings)
- Cấu hình hệ thống, cài đặt người dùng và thông báo

### i) Landing Page

- LandingPage.tsx (/)
- Màn hình giới thiệu và điều hướng vào hệ thống

## II. Phân tích nghiệp vụ

### 1. Mô tả bài toán

Trong cuộc sống hiện nay, việc theo dõi và quản lý chi tiêu là nhu cầu thiết yếu của sinh viên và người đi làm. Tuy nhiên, nhiều người không có thói quen ghi chép rõ ràng, dẫn đến tình trạng chi tiêu không kiểm soát, không biết tiền đã tiêu vào những khoản nào hoặc liệu tháng này có vượt ngân sách hay không.

Bài toán đặt ra là cần có một hệ thống cho phép người dùng:

- Ghi nhận các giao dịch hàng ngày
- Quản lý nhiều ví/ tài khoản cùng một lúc
- Theo dõi ngân sách theo tháng
- Xem báo cáo tài chính trực quan
- Nhận gợi ý tối ưu chi tiêu nhờ AI

Hệ thống Vissmart được xây dựng để giải quyết những vấn đề trên bằng cách cung cấp một nền tảng quản lý thu chi cá nhân đầy đủ, dễ sử dụng và phù hợp với thói quen của người dùng Việt Nam.

### 2. Quy trình nghiệp vụ chung

Quy trình nghiệp vụ chung của hệ thống được mô tả qua các bước sau:

#### a) Đăng ký/ Đăng nhập

- Người dùng tạo tài khoản hoặc đăng nhập hệ thống để nhận JWT Token.
- Backend sử dụng Spring Security kết hợp với JwtService để xác thực và gắn thông tin người dùng vào các request tiếp theo.

#### b) Quản lý ví

- Người dùng tạo các loại ví như ví tiền mặt, thẻ ngân hàng hoặc ví điện tử và khai báo số dư ban đầu.
- Dữ liệu được lưu trong bảng wallets thông qua entity Wallet và WalletRepository.

#### c) Ghi nhận giao dịch

- Người dùng thêm giao dịch thu/ chi. Hệ thống tự cập nhật số dư tương ứng.

#### d) Thiết lập ngân sách

- Người dùng đặt hạn mức chi tiêu theo từng danh mục trong tháng.
- Hệ thống theo dõi mức chi và cộng dồn vào trường spent trong entity Budget dựa trên các giao dịch thực tế.

#### e) Xem báo cáo và phân tích

- Hệ thống hiển thị biểu đồ, thống kê và xu hướng chi tiêu theo tháng/ năm.
- Phần này được backend xử lý thông qua AnalyticsService và AnalyticsRepository, trả dữ liệu về cho các trang như ExpenseAnalyticsPage.tsx, ReportsPage.tsx.

#### f) Chatbot AI hỗ trợ

- Người dùng có thể hỏi đáp về tài chính, nhận gợi ý chi tiêu.
- Frontend sử dụng geminiService.ts để kết nối tới Gemini API, đồng thời backend lưu lịch sử chat vào bảng ai\_chat\_history thông qua entity ChatHistory.

**g) Quản lý cài đặt cá nhân**

- Bao gồm tùy chỉnh thông báo, xem thông tin người dùng, quản lý danh mục chi tiêu (Category) thông qua module Settings.

**3. Use Case tổng thể hệ thống**

- Hệ thống Vissmart có một tác nhân duy nhất là người dùng (User)
- Người dùng có thể thực hiện các chức năng sau:
  - o Đăng ký tài khoản
  - o Đăng nhập và nhận JWT Token để xác thực
  - o Quản lý ví tiền
  - o Quản lý giao dịch
  - o Lọc và xem lịch sử giao dịch theo thời gian, ví hoặc danh mục
  - o Quản lý ngân sách
  - o Theo dõi mức chi và cảnh báo vượt ngân sách
  - o Xem báo cáo và phân tích tài chính
  - o Trao đổi với Chatbot AI
  - o Quản lý cài đặt cá nhân

**4. Use Case theo từng module**

**a) Authentication Module**

- Đăng ký tài khoản
- Đăng nhập
- Xác thực người dùng bằng JWT

**b) Wallet Management Module**

- Tạo ví mới
- Chỉnh sửa thông tin ví
- Xóa ví
- Xem danh sách ví theo user

**c) Transaction Management Module**

- Thêm giao dịch thu/chi
- Sửa giao dịch
- Xóa giao dịch
- Lọc giao dịch
- Xem giao dịch theo danh mục hoặc theo ví

**d) Budget Management Module**

- Tạo ngân sách theo danh mục
- Xem hạn mức và mức chi đã dùng
- Cảnh báo vượt ngân sách

**e) Analytics & Reporting Module**

- Xem báo cáo tài chính
- Xem biểu đồ chi tiêu
- Phân tích xu hướng
- Xuất báo cáo (PDF/CSV nếu có)

**f) AI Chat Module**

- Gửi câu hỏi đến AI
- Nhận phản hồi

- Lưu lịch sử chat
- g) Settings Module**
  - Tùy chỉnh danh mục chi tiêu
  - Tùy chỉnh thông báo
  - Xem và cập nhật thông tin người dùng

### III. Công nghệ và kiến trúc hệ thống

#### 1. Công nghệ sử dụng

Hệ thống Vissmart được xây dựng với bộ công nghệ hiện đại, tách biệt Frontend – Backend theo mô hình RESTful API:

##### a) Backend – Spring Boot

- Spring Boot 3 – tạo ứng dụng Java nhanh chóng, cấu hình tối thiểu
- Spring Data JPA – thao tác cơ sở dữ liệu thông qua Repository
- Spring Security – bảo mật & phân quyền
- JWT (JSON Web Token) – xác thực stateless
- Lombok – giảm boilerplate code
- JJWT – xử lý JWT
- MySQL Connector – kết nối cơ sở dữ liệu

##### b) Frontend – React + Vite

- ReactJS – xây giao diện người dùng
- Vite – công cụ build nhanh, hiện đại
- Axios – gọi API
- React Router DOM – điều hướng trang
- MUI (Material UI) – component UI
- Chart.js hoặc Recharts – hiển thị biểu đồ
- LocalStorage – lưu token đăng nhập

##### c) Cơ sở dữ liệu – MySQL 8

- Lưu người dùng, giao dịch, ví, ngân sách, danh mục, báo cáo, lịch sử chat.
- Sử dụng khóa ngoại, index, và ON DELETE CASCADE.
- ERD gồm 10 bảng, thiết kế thống nhất và mở rộng được.

#### 2. Kiến trúc tổng thể hệ thống

##### a) Các thành phần chính của kiến trúc:

- **Frontend (React):** Đảm nhiệm việc hiển thị giao diện, cho phép người dùng thao tác với các chức năng như quản lý ví, giao dịch, ngân sách, báo cáo và chatbot AI. Frontend sử dụng Axios để gọi API và lưu JWT trong LocalStorage để xác thực.
- **Backend (Spring Boot):** Xử lý toàn bộ nghiệp vụ hệ thống thông qua các lớp Service và Controller, bao gồm xác thực người dùng, quản lý ví, giao dịch, ngân sách, danh mục, báo cáo và lịch sử chat. Backend cũng chịu trách nhiệm xác thực JWT và kiểm soát quyền truy cập.
- **Database (MySQL):** Lưu trữ dữ liệu toàn hệ thống như thông tin người dùng, ví, giao dịch, ngân sách, danh mục, thông báo, lịch sử chat AI và cài đặt người dùng. Spring Data JPA cho phép thao tác với DB thông qua Repository.
- **AI Service (Gemini API):** Hỗ trợ chatbot bằng cách xử lý câu hỏi người dùng và trả lời thông minh. Backend lưu lịch sử chat vào cơ sở dữ liệu.



- **Email Service (SendGrid/SMTP):** Dùng để gửi email đặt lại mật khẩu trong module Authentication.

**b) Luồng hoạt động tổng quát của hệ thống:**

- Người dùng thao tác trên giao diện React
- Frontend gửi yêu cầu đến Backend thông qua REST API
- Backend xác thực JWT
- Backend chuyển yêu cầu đến các lớp Service để xử lý nghiệp vụ
- Service tương tác với Repository để truy xuất hoặc cập nhật cơ sở dữ liệu
- Backend trả về dữ liệu dạng JSON
- Frontend nhận dữ liệu và hiển thị lên giao diện

**c) Mô tả kiến trúc hệ thống :**

Kiến trúc tổng thể được tổ chức theo mô hình nhiều lớp:

- Client Layer (React)
- API Layer (Controller)
- Service Layer (xử lý logic)
- Repository Layer (truy vấn DB)
- Database Layer (MySQL)
- External Services (Gemini AI, SendGrid Email)

Cách tổ chức phân lớp giúp hệ thống dễ mở rộng, dễ bảo trì và đảm bảo tính đóng gói theo nguyên tắc của lập trình hướng đối tượng.

## IV. Thiết kế hướng đối tượng

### 1. Tổng quan 4 tính chất OOP

Hệ thống Vissmart được xây dựng theo tư duy lập trình hướng đối tượng, thể hiện thông qua bốn tính chất cơ quan:

**a) Tính đóng gói**

Các lớp trong backend đều được thiết kế để đóng gói dữ liệu và hành vi liên quan:

- Các model như User, Wallet, Transaction, Budget, ChatHistory, Category đều sử dụng thuộc tính private, được truy cập thông qua getter/ setter.
- Login xử lý được gom vào Service Layer như AuthService, WalletService, TransactionService, tránh để Controller xử lý trực tiếp.
- Một số lớp sử dụng @PrePersist (ví dụ: ChatHistory, Category) để tự động gán giá trị khởi tạo, đây là đóng gói hành vi bên trong đối tượng.

**b) Tính trừu tượng**

Hệ thống ẩn đi các chi tiết phức tạp:

- Repository chỉ cần kế thừa JpaRepository → không cần viết SQL.
- Interface AnalyticsService định nghĩa các hành vi phân tích, tách biệt với lớp triển khai AnalyticsServiceImpl.
- JwtService trừu tượng hóa toàn bộ logic tạo / kiểm tra JWT.

**c) Tính kế thừa**

Tính kế thừa thể hiện rõ trong:

- Các Repository kế thừa JpaRepository<..., ...> → tự động có CRUD mà không phải viết lại.
- Lớp User kế thừa từ UserDetails của Spring Security → override các phương thức như getAuthorities(), isEnabled(),... để phục vụ cơ chế bảo mật.
- Các Exception Handler kế thừa từ cấu trúc Spring MVC để tùy biến thông điệp lỗi toàn hệ thống.

#### d) Tính đa hình

- Interface AnalyticsService có thể được triển khai bởi nhiều lớp khác nhau trong tương lai.
- Lớp User khi được Spring Security sử dụng sẽ hoạt động như một UserDetails, thay đổi hành vi theo context.
- Các phương thức của Repository như findById(...) được Spring Data JPA xử lý tự động dựa trên đa hình động.

## 2. Áp dụng OOP vào Spring Boot

### a) Model Layer

#### Các entity chính:

- User
- Wallet
- Transaction
- Category
- Budget
- ChatHistory
- Report
- Notifications
- UserPreferences

#### Đặc điểm:

- Mỗi entity là một đối tượng của bài toán thực tế (ví dụ: Wallet là một ví của người dùng).
- Dùng annotation @Entity, @Table, @Id,... để ánh xạ với database.
- Đóng gói dữ liệu: thuộc tính private + getter/setter.
- Một số entity có logic riêng như @PrePersist để tự điền dữ liệu ban đầu.

### b) Repository Layer

#### Các Repository như:

- o UserRepository
- o WalletRepository
- o TransactionRepository
- o BudgetRepository
- o CategoryRepository
- o ChatHistoryRepository
- o AnalyticsRepository

đều kế thừa JpaRepository.

#### Lợi ích OOP:

- o Không phải viết SQL thủ công → hệ thống trừu tượng hóa truy cập dữ liệu.
- o Mỗi Repository đại diện cho kho dữ liệu của một kiểu đối tượng.

### c) Service Layer

- Đây là tầng quan trọng nhất của backend, xử lý toàn bộ nghiệp vụ:
  - o AuthService – đăng ký, đăng nhập
  - o WalletService – tạo ví, cập nhật số dư
  - o TransactionService – thêm giao dịch, cập nhật budget và wallet
  - o BudgetService – quản lý ngân sách
  - o AnalyticsServiceImpl – xử lý phân tích dữ liệu
  - o ChatHistoryService – lưu và truy vấn lịch sử chat
  - o UserSettingsService – cài đặt cá nhân hóa

- Luồng nghiệp vụ Phức tạp (điểm mạnh trong OOP):  
Ví dụ: Khi tạo giao dịch → TransactionService sẽ:  
Bước 1 - Lưu giao dịch  
Bước 2 - Cập nhật số dư ví (WalletService.updateBalance())  
Bước 3 - Cập nhật mức chi tiêu trong ngân sách (BudgetRepository)  
Bước 4 - Trả dữ liệu về FE
- d) Controller Layer**
  - Các Controller như AuthController, WalletController, TransactionController, BudgetController, ChatHistoryController chỉ làm nhiệm vụ:
    - o Nhận request
    - o Gọi service
    - o Trả response
  - Controller không xử lý nghiệp vụ, giúp phân tách rõ ràng các tầng trong OOP.

### 3. Mô hình lớp

Mô tả các nhóm lớp chính:

#### a) Nhóm lớp Người dùng (User Domain)

- User
- UserRepository
- AuthService, JwtService, EmailService
- AuthController
- Nhóm xử lý xác thực bằng JWT + mã hóa mật khẩu + reset password.

#### b) Nhóm lớp Ví (Wallet Domain)

- Wallet
- WalletRepository
- WalletService
- WalletController
- Chứa thông tin ví tiền, tài khoản ngân hàng, ví điện tử.

#### c) Nhóm lớp Giao dịch (Transaction Domain)

- Transaction
- TransactionRepository
- TransactionService
- TransactionController
- Service này liên kết 3 domain khác:
- Cập nhật ví → WalletService
- Cập nhật ngân sách → BudgetRepository
- Gọi Category → CategoryRepository

#### d) Nhóm lớp Ngân sách (Budget Domain)

- Budget
- BudgetRepository
- BudgetService
- BudgetController

#### e) Nhóm lớp Danh mục (Category Domain)

- Category
- CategoryRepository
- SettingsService / CategoryService

- SettingsController
- f) Nhóm lớp Phân tích (Analytics Domain)**
  - AnalyticsRepository
  - AnalyticsService (Interface)
  - AnalyticsServiceImpl (Implement)
  - DTO: ChartDataDTO, FinancialReportDTO, TrendDTO
  - Nhóm này xử lý thống kê, báo cáo, biểu đồ.
- g) Nhóm lớp Chat AI (ChatHistory Domain)**
  - ChatHistory
  - ChatHistoryRepository
  - ChatHistoryService
  - ChatHistoryController
  - Entity sử dụng @PrePersist để tự gắn createdAt, isDeleted
  - Cho phép soft delete

#### 4. Design Pattern áp dụng

##### a) Repository Pattern

- Toàn bộ lớp Repository kế thừa JpaRepository. Đây là một trong những pattern quan trọng nhất trong Spring Data JPA, giúp tách biệt logic dữ liệu và nghiệp vụ.

##### b) Service Layer Pattern

- Tầng Service tập trung toàn bộ nghiệp vụ:
  - o Update balance
  - o Cập nhật ngân sách
  - o Validate giao dịch
  - o Gửi mail
  - o Sinh JWT
- Giúp Controller nhẹ, dễ bảo trì, đúng chuẩn MVC.

##### c) DTO Pattern

- Các DTO như:
  - o LoginRequest, RegisterRequest, AuthResponse
  - o ChartDataDTO, TrendDTO
  - o ExportResponse
- Giúp tách lớp Entity ra khỏi dữ liệu trả về API → tăng bảo mật và dễ mở rộng.

##### d) Strategy/ Abstraction Pattern trong Analytics

- AnalyticsService là interface, AnalyticsServiceImpl implement.

##### e) Singleton Pattern trong các Service

- Trong Spring: Các Service (@Service) và Repository (@Repository) mặc định là Singleton – một instance duy nhất trong toàn bộ ứng dụng.

##### f) Factory Pattern

- Cơ chế tạo JWT trong JwtService chính là một dạng Factory:
  - o Nhận Input – tạo token theo cấu trúc chuẩn.
  - o Giúp Controller không phải biết cách token được tạo ra.

## V. Thiết kế cơ sở dữ liệu

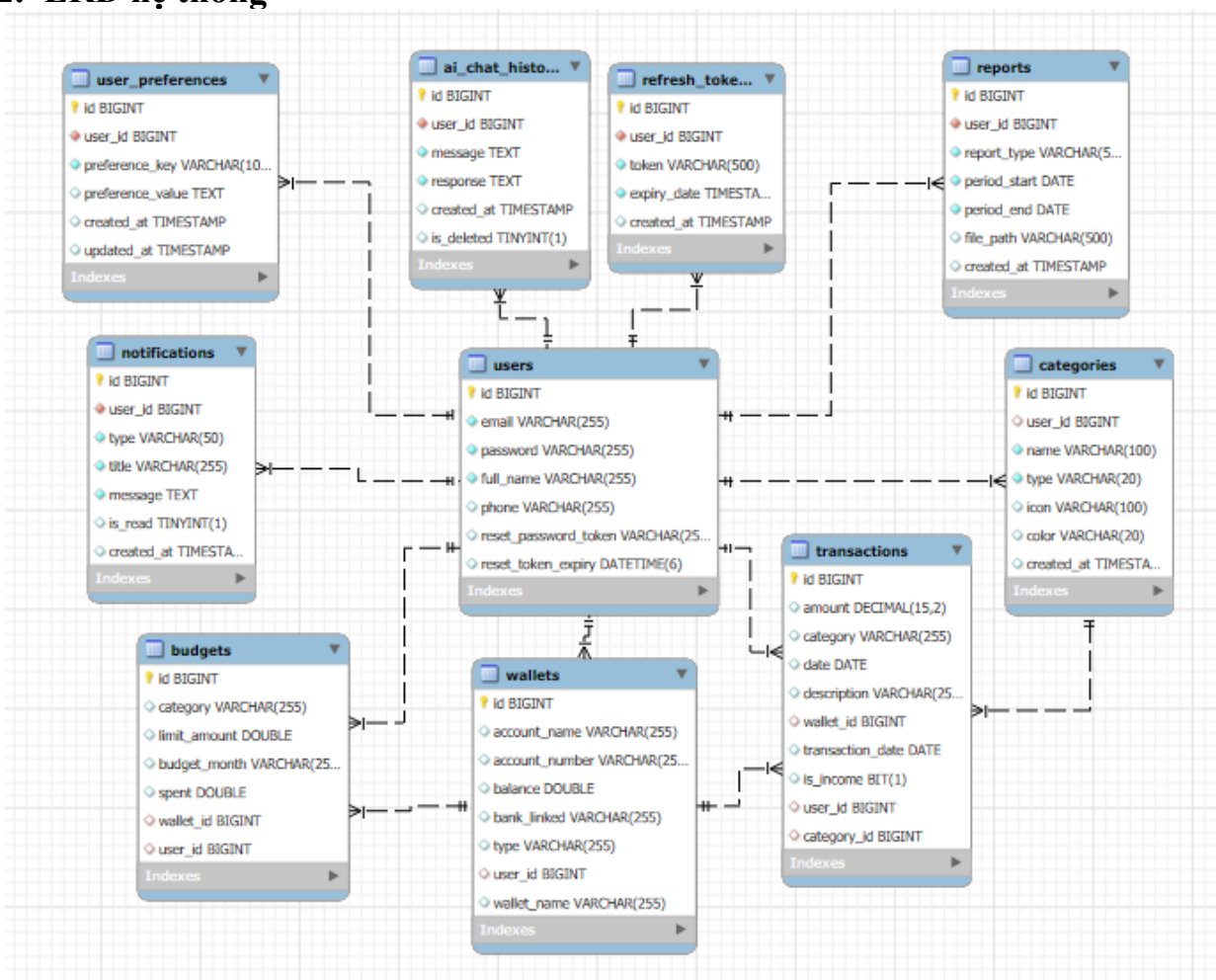
### 1. Phân tích dữ liệu

Hệ thống Vissmart là một ứng dụng quản lý thu chi cá nhân, do đó dữ liệu cần lưu trữ gồm:

- Thông tin người dùng
- Danh mục chi tiêu
- Ví/ tài khoản
- Giao dịch thu – chi
- Ngân sách theo tháng
- Báo cáo tài chính
- Lịch sử chat AI
- Thông báo
- Tùy chọn người dùng

Các bảng đều có quan hệ dựa trên user\_id để đảm bảo mỗi người dùng chỉ quản lý dữ liệu của riêng mình, đúng với mô hình non-admin.

## 2. ERD hệ thống



Hình 5.1 – Sơ đồ ERD của hệ thống Vissmart

Sơ đồ ERD mô tả mối quan hệ giữa các bảng chính như sau:

- Một User có thể sở hữu nhiều Wallet, Transaction, Budget, Category, Thông báo, Lịch sử chat và User Preferences → quan hệ 1 – N.
- Mỗi Wallet có thể chứa nhiều Transaction.
- Mỗi Category có thể gắn với nhiều Transaction.
- Budget có thể ràng buộc với một ví cụ thể hoặc áp dụng toàn hệ thống.

Quan hệ được mô tả đầy đủ:

User (1) ----- (N) Wallet  
 User (1) ----- (N) Transaction  
 User (1) ----- (N) Category  
 User (1) ----- (N) Budget  
 Wallet (1) ----- (N) Transaction  
 Category (1) ----- (N) Transaction  
 User (1) ----- (N) AI Chat History  
 User (1) ----- (N) Notification  
 User (1) ----- (N) User Preference

### 3. Thiết kế bảng

Cơ sở dữ liệu được thiết kế theo mô hình quan hệ. Trong đó, bảng users đóng vai trò trung tâm, liên kết với các bảng phụ thông qua khóa ngoại user\_id. Gồm các bảng sau:

#### a) Bảng users

- Bảng users lưu trữ thông tin tài khoản người dùng, bao gồm email, mật khẩu, họ tên, số điện thoại, token đặt lại mật khẩu và thời gian hết hạn.

#### b) Bảng wallets

- Bảng wallets lưu thông tin các ví hoặc tài khoản mà người dùng sở hữu. Mỗi ví chứa tên ví, loại ví (tiền mặt, ngân hàng, ví điện tử), số dư và thông tin liên kết ngân hàng. Một người dùng có thể sở hữu nhiều ví khác nhau.

#### c) Bảng transactions

- Bảng transactions ghi nhận toàn bộ giao dịch thu/ chi của người dùng, bao gồm số tiền, ngày giao dịch, mô tả, loại giao dịch (thu hoặc chi), ví sử dụng và danh mục liên quan. Đây là bảng có tần suất đọc/ghi cao nhất trong hệ thống.

#### d) Bảng categories

- Bảng categories lưu danh mục thu/ chi, mỗi danh mục thuộc một người dùng và được gắn với icon, màu sắc để thể hiện trực quan trên giao diện. Danh mục được sử dụng để phân loại giao dịch và phân tích chi tiêu.

#### e) Bảng budgets

- Bảng budgets lưu ngân sách theo tháng của từng danh mục. Mỗi ngân sách bao gồm hạn mức chi tiêu (limit\_amount), số tiền đã tiêu (spent) và tháng áp dụng (budget\_month). Mỗi budget có thể áp dụng cho một ví cụ thể hoặc cho toàn bộ ví của người dùng.

#### f) Bảng refresh\_tokens

- Bảng refresh\_tokens lưu trữ token làm mới phiên đăng nhập, phục vụ cơ chế xác thực JWT giúp người dùng duy trì đăng nhập an toàn mà không cần nhập lại mật khẩu.

#### g) Bảng reports

- Bảng reports lưu thông tin các báo cáo tài chính đã được hệ thống sinh ra, bao gồm loại báo cáo, khoảng thời gian và đường dẫn file báo cáo trên hệ thống.

#### h) Bảng ai\_chat\_history

- Bảng này lưu lịch sử tương tác giữa người dùng và hệ thống AI. Mỗi bản ghi chứa tin nhắn người dùng, phản hồi AI, thời gian tạo và trạng thái đã xóa/ẩn. Đây là dữ liệu phục vụ cho module Chatbot AI.

#### i) Bảng notifications

- Bảng notifications lưu các thông báo gửi đến người dùng, bao gồm tiêu đề, nội dung, loại thông báo và trạng thái đã đọc. Bảng này hỗ trợ việc nhắc người dùng về các sự kiện quan trọng như vượt ngân sách hoặc báo cáo tháng mới.

#### j) Bảng user\_preferences

- Bảng này lưu trữ các tùy chọn cá nhân hóa của người dùng, ví dụ như chủ đề giao diện, ngôn ngữ hoặc cài đặt hiển thị. Ràng buộc (user\_id, preference\_key) đảm bảo mỗi người dùng chỉ có một giá trị duy nhất cho mỗi loại tùy chọn.

#### 4. Ràng buộc và mối quan hệ dữ liệu

Hệ thống sử dụng các ràng buộc khóa ngoại (Foreign Key) để đảm bảo tính toàn vẹn dữ liệu. Cụ thể:

- user\_id là khóa ngoại xuất hiện hầu hết trong các bảng → đảm bảo mỗi dữ liệu thuộc về một người dùng cụ thể.
- Bảng wallets, transactions, budgets, categories, notifications, ai\_chat\_history, user\_preferences sử dụng chế độ ON DELETE CASCADE, giúp dữ liệu liên quan được tự động xóa khi người dùng bị xóa.
- wallet\_id trong bảng transactions và budgets đảm bảo giao dịch/ngân sách phải thuộc một ví hợp lệ.
- category\_id trong bảng transactions liên kết đến bảng categories giúp phân loại giao dịch chính xác.
- Hầu hết các khóa ngoại đều có INDEX hỗ trợ truy vấn nhanh hơn trong các màn hình thống kê, tra cứu và dashboard.

Những ràng buộc này giúp cơ sở dữ liệu hoạt động ổn định, ngăn ngừa các lỗi thường gặp như giao dịch không thuộc ví nào hoặc danh mục bị xóa khi vẫn còn giao dịch liên quan.

#### 5. Bảng mẫu

Một vài bảng mẫu:

- Bảng users:

|   | id   | email            | password                                     | full_name | phone      | reset_password_token | reset_token_expiry |
|---|------|------------------|----------------------------------------------|-----------|------------|----------------------|--------------------|
| ▶ | 1    | user@example.com | \$2a\$10\$bXKQDoHIpHKD6sq0ryjAfeFjaIWgZFO... | User      | 0123456789 | NULL                 | NULL               |
|   | 2    | abc@gmail.com    | \$2a\$10\$BEHqK6ocaKeKnLYWOBlo3u66TQACVf...  | Adun      | 087472973  | NULL                 | NULL               |
| * | NULL | NULL             | NULL                                         | NULL      | NULL       | NULL                 | NULL               |

- Bảng wallets:

|   | id   | account_name | account_number | balance  | bank_linked | type     | user_id | wallet_name |
|---|------|--------------|----------------|----------|-------------|----------|---------|-------------|
| ▶ | 1    | Ví 1         | 0198203999     | 1000000  | Momo        | E-Wallet | 2       | Ví 1        |
|   | 2    | NULL         | NULL           | 4000000  | Cash        | Cash     | 2       | Ví 2        |
|   | 3    | NGUYỄN VĂN A | 09809809802342 | 10000... | VietinBank  | Bank     | 2       | Ví 3        |
| * | NULL | NULL         | NULL           | NULL     | NULL        | NULL     | NULL    | NULL        |

- Bảng budgets:

|   | id   | category | limit_amount | budget_month | spent   | wallet_id | user_id |
|---|------|----------|--------------|--------------|---------|-----------|---------|
| ▶ | 1    | Nhà cửa  | 3000000      | 2025-11      | 2000000 | NULL      | 2       |
|   | 2    | Ăn uống  | 2000000      | 2025-11      | 30000   | NULL      | 2       |
|   | 3    | Hóa đơn  | 1000000      | 2025-11      | 0       | NULL      | 2       |
| * | NULL | NULL     | NULL         | NULL         | NULL    | NULL      | NULL    |

- Bảng transactions:

|   | id   | amount      | category | date | description | wallet_id | transaction_date | is_income | user_id | category_id |
|---|------|-------------|----------|------|-------------|-----------|------------------|-----------|---------|-------------|
| ▶ | 1    | 30000.00    | Ăn uống  | NULL | Ăn sáng     | 1         | 2025-11-27       | 0         | 2       | NULL        |
|   | 2    | 2000000.00  | Nhà cửa  | NULL | Tiền nhà    | 1         | 2025-11-27       | 0         | 2       | NULL        |
|   | 3    | 10000000.00 | Thu nhập | NULL | Lương       | 3         | 2025-11-27       | 1         | 2       | NULL        |
| * | NULL | NULL        | NULL     | NULL | NULL        | NULL      | NULL             | NULL      | NULL    | NULL        |

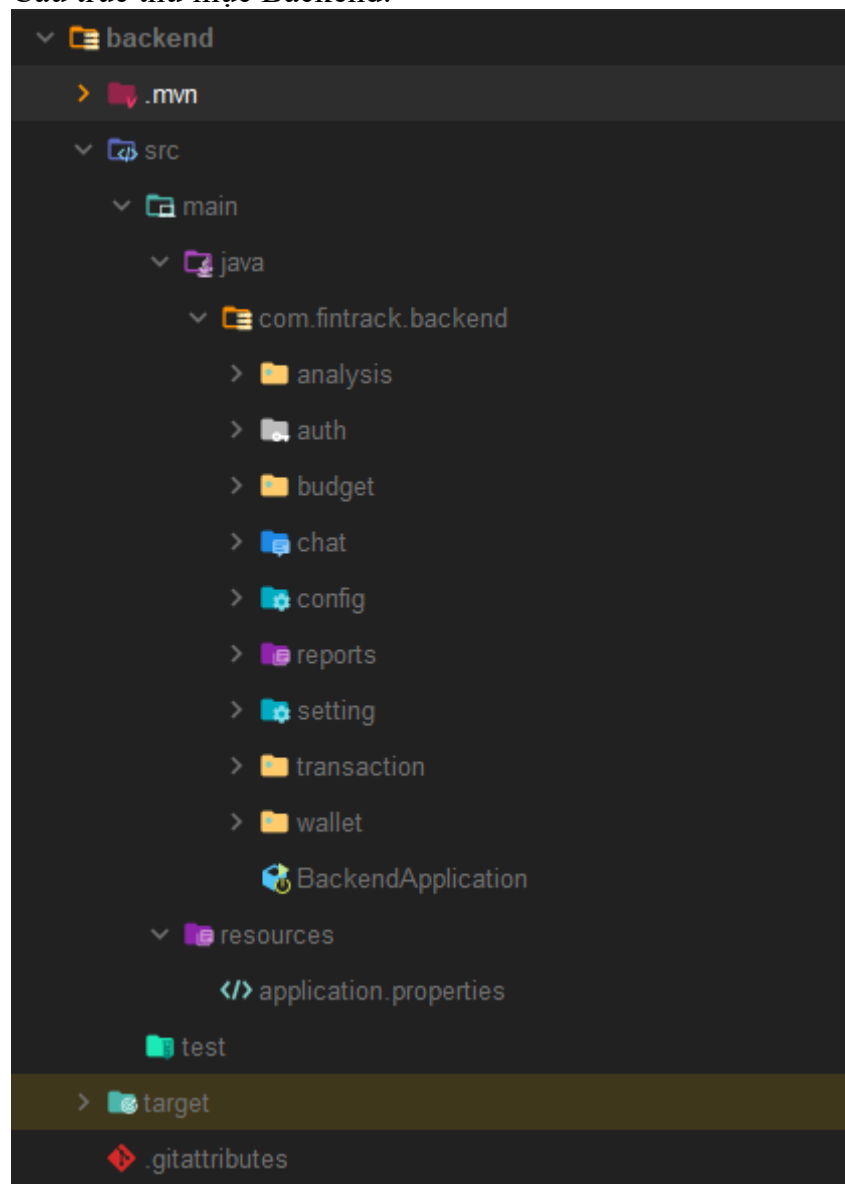
## VI. Khai triển hệ thống

### 1. Tổ chức module của hệ thống

Hệ thống Vissmart được khai triển dựa trên kiến trúc đa tầng và phân chia module rõ ràng theo từng chức năng. Cả Backend và Frontend đều được tách thành các module độc lập nhằm đảm bảo dễ bảo trì, dễ mở rộng và tuân thủ nguyên tắc đóng gói của OOP.

#### a) Backend (Spring Boot)

- Cấu trúc thư mục Backend:





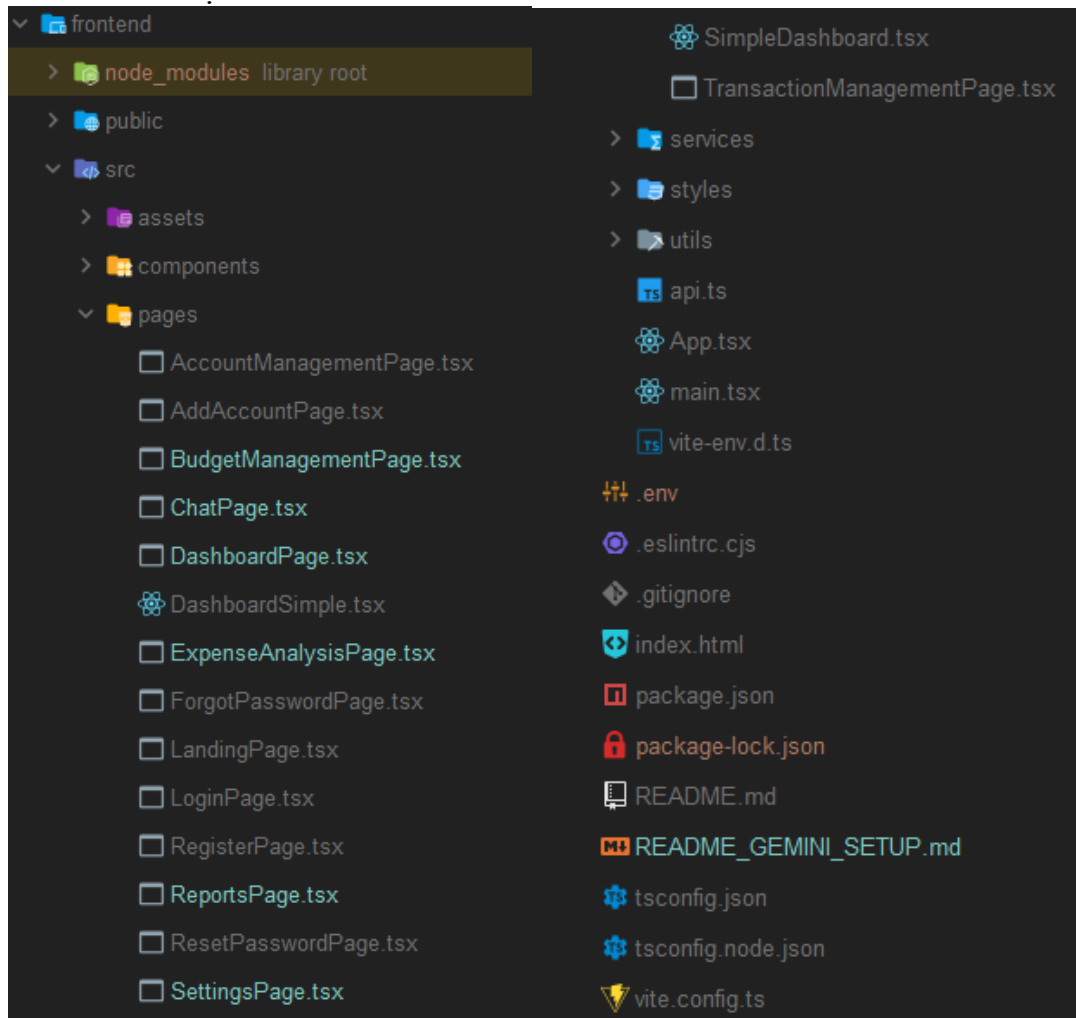
- Backend được chia thành 9 module chính:

| Module              | Chức năng                                             |
|---------------------|-------------------------------------------------------|
| <b>auth</b>         | Đăng ký, đăng nhập, cấp phát JWT, xác thực người dùng |
| <b>wallet</b>       | Quản lý ví, số dư, loại ví                            |
| <b>transaction</b>  | Ghi nhận giao dịch thu/chi, lọc giao dịch             |
| <b>budget</b>       | Hạn mức chi tiêu, theo dõi số tiền đã chi theo tháng  |
| <b>category</b>     | Quản lý danh mục thu/chi của người dùng               |
| <b>analytics</b>    | Tính toán báo cáo, thống kê, tổng hợp chi tiêu        |
| <b>ai</b>           | Chatbot tài chính, lưu lịch sử chat                   |
| <b>notification</b> | Gửi email, thông báo hệ thống                         |
| <b>config</b>       | JWT Filter, Spring Security, CORS, các cấu hình chung |

- Backend tổ chức module theo controller → service → repository → entity, tuân thủ chuẩn mô hình 4 tầng (presentation – service – data – model).

## b) Frontend (React)

- Cấu trúc thư mục Frontend:



- Frontend được chia thành page/ module:

| Module giao diện         | Chức năng                                  |
|--------------------------|--------------------------------------------|
| <b>Auth Pages</b>        | Đăng ký, đăng nhập                         |
| <b>Wallet Pages</b>      | Tạo ví, xem số dư, chỉnh sửa ví            |
| <b>Transaction Pages</b> | Thêm, sửa, lọc giao dịch                   |
| <b>Budget Pages</b>      | Tạo ngân sách, theo dõi chi tiêu           |
| <b>Analytics Pages</b>   | Biểu đồ tài chính, thống kê theo tháng/năm |
| <b>AI Chat Pages</b>     | Chat Gemini AI, gợi ý tài chính            |
| <b>Settings Pages</b>    | Danh mục, thông báo, thông tin cá nhân     |

- Frontend sử dụng:
  - React + Vite
  - Axios để call API
  - Zustand để quản lý trạng thái
  - Material UI để tạo giao diện

## 2. Cách các module được khởi tạo và hoạt động

### a) Backend

- Khi backend chạy, Spring Boot tự động load các module theo pipeline:
  - Spring Boot khởi động → load ApplicationConfig, SecurityConfig.
  - Spring Security tạo filter chain → đăng ký JwtAuthenticationFilter.
  - Hibernate quét các entity (User, Wallet, Transaction, Budget...) và map với MySQL.
  - Các **controller** được ánh xạ vào các API:
    - /api/auth/\*\*
    - /api/wallets/\*\*
    - /api/transactions/\*\*
    - ...
  - Service layer được inject tự động với @RequiredArgsConstructor, xử lý nghiệp vụ.
  - Repository layer giao tiếp DB bằng JPA.
- Mỗi module hoạt động độc lập nhưng liên kết thông qua OOP (entity – service – repository).

### b) Frontend

- Khi frontend chạy (npm run dev):
  - React khởi tạo root app → Load Zustand store.
  - React Router điều hướng giữa các trang.
  - Các component gọi API qua Axios.
  - Kết quả backend trả về → hiển thị bằng MUI components & biểu đồ.
  - Với module AI, FE gọi service geminiService.ts.

## 3. Backend – Frontend giao tiếp

- Giao tiếp thông qua REST API chuẩn JSON.
- Luồng chung:
  - FE gửi request (Axios) → Backend /api/...
  - Backend kiểm tra JWT:
    - Nếu request là /auth/login, /auth/register → không cần token

- Các API khác → bắt buộc token
- Controller nhận request → Service → Repository → DB
- Response trả về dạng JSON
- FE parse và hiển thị trên UI
- Ví dụ luồng giao tiếp khi tạo giao dịch:
  - FE → POST /api/transactions
  - Backend:
    - Controller nhận → TransactionService xử lý
    - Kiểm tra số dư ví
    - Lưu Transaction vào DB
    - Trừ tiền ví
    - Nếu có Budget → cập nhật spent
  - Backend → trả JSON kết quả về FE.

#### 4. Một vài API mẫu

##### - Đăng nhập:

POST <http://localhost:8080/api/auth/login>

```
{
  "email": "abc@gmail.com",
  "password": "abc@123"
}
```

##### - Tạo giao dịch:

POST <http://localhost:8080/api/transactions>

```
{
  "id": 1,
  "date": "2025-11-27",
  "description": "Ăn sáng ",
  "amount": 30000.00,
  "category": "Ăn uống",
  "categoryId": null,
  "userId": 2,
  "walletId": 1,
  "income": false,
  "isIncome": false
}
```

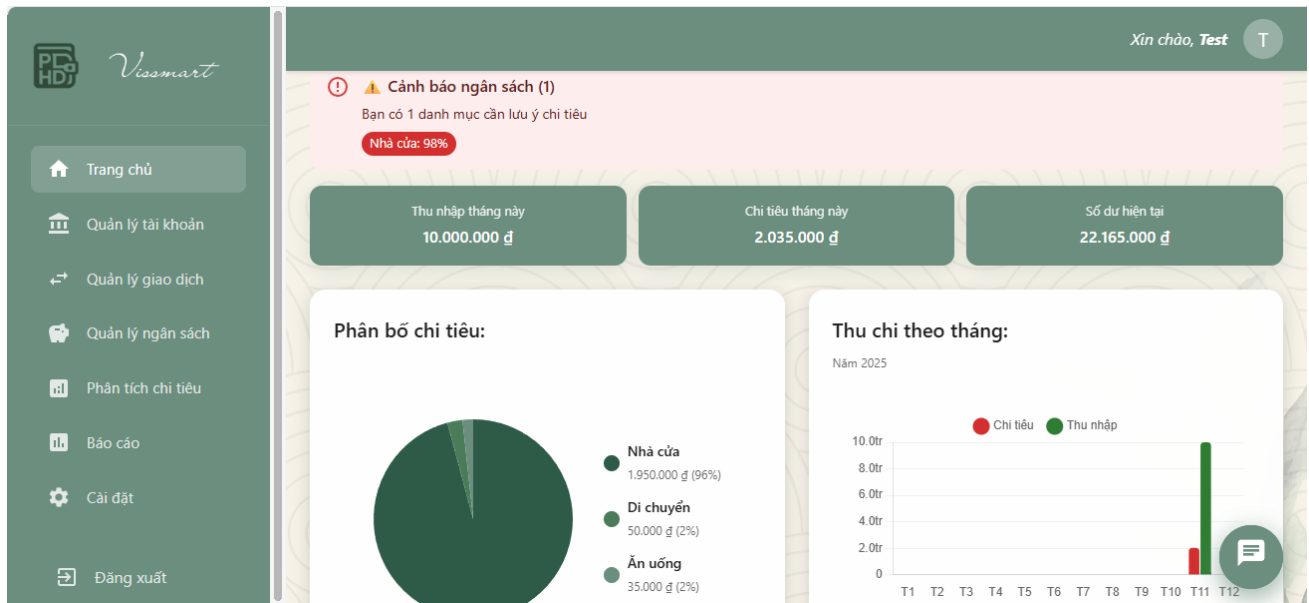
##### - Tạo ngân sách:

POST <http://localhost:8080/api/budgets>

```
{
  "id": 1,
  "category": "Nhà cửa",
  "limitAmount": 3000000.0,
  "spent": 2000000.0,
  "month": "2025-11",
  "userId": 2
}
```

## 5. Ảnh giao diện chính

- Trang Dashboard tổng quan:



- Trang quản lý ví:



- Trang quản lý giao dịch:

- Trang chủ
- Quản lý tài khoản
- Quản lý giao dịch**
- Quản lý ngân sách
- Phân tích chi tiêu
- Báo cáo
- Cài đặt
- Đăng xuất

Xin chào, Test

### Quản lý giao dịch

+ THÊM GIAO DỊCH

| Thời gian  | Nội dung | Số tiền       | Danh mục  | Vị   | Hành động |
|------------|----------|---------------|-----------|------|-----------|
| 28/11/2025 | Tiền trợ | -1.950.000 đ  | Nhà cửa   | Vị 2 |           |
| 28/11/2025 | Đổ xăng  | -50.000 đ     | Di chuyển | Vị 1 |           |
| 28/11/2025 | Ăn sáng  | -35.000 đ     | Ăn uống   | Vị 3 |           |
| 28/11/2025 | Lương    | +10.000.000 đ | Thu nhập  | Vị 2 |           |

- Trang quản lý ngân sách:

- Trang chủ
- Quản lý tài khoản
- Quản lý giao dịch
- Quản lý ngân sách**
- Phân tích chi tiêu
- Báo cáo
- Cài đặt
- Đăng xuất

Xin chào, Test

### Cảnh báo ngân sách (1)

**Nhà cửa - Sắp vượt ngân sách (98%)**  
 Đã chi: 1.950.000 đ    Ngân sách: 2.000.000 đ  

98% đã sử dụng

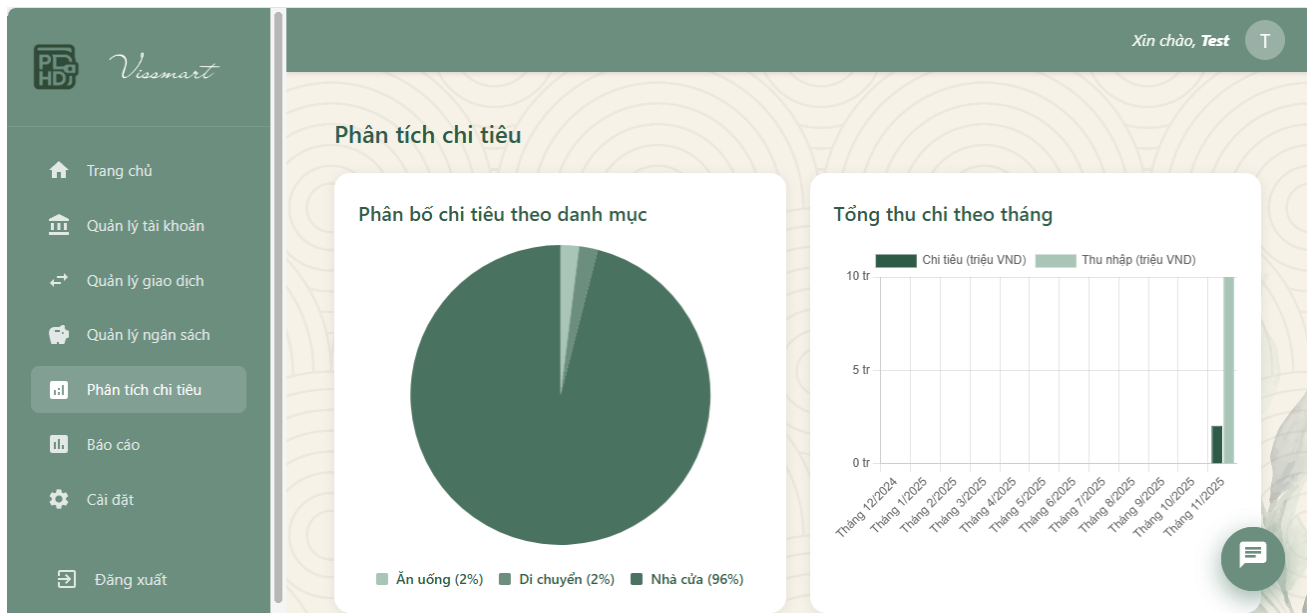
Gợi ý: Còn 50.000 đ. Hãy chi tiêu cẩn thận!

Tháng Mười Một 2025

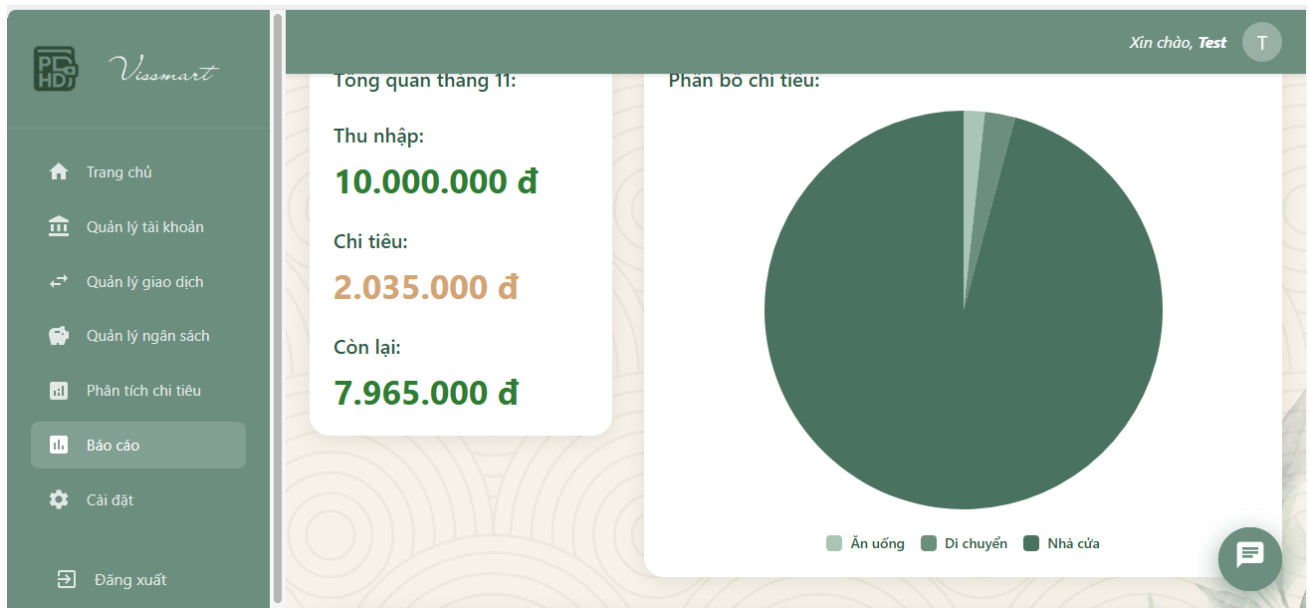
+ Tạo ngân sách mới

| Danh mục  | Hạn mức     | Đã chi (Tự động) | Còn lại     | Tháng   | Hành động |
|-----------|-------------|------------------|-------------|---------|-----------|
| Ăn uống   | 2.000.000 đ | 35.000 đ         | 1.965.000 đ | 2025-11 |           |
| Di chuyển | 300.000 đ   | 50.000 đ         | 250.000 đ   | 2025-11 |           |
| Nhà cửa   | 2.000.000 đ | 1.950.000 đ      | 50.000 đ    | 2025-11 |           |

- Trang phân tích chi tiêu:



- Trang báo cáo:



- Trang cài đặt:

**Thông tin cá nhân**

Họ và tên  
Test

Email  
abcd@gmail.com

Số điện thoại  
0898789352

LƯU THAY ĐỔI

**Thay đổi mật khẩu**

Mật khẩu hiện tại

Mật khẩu mới

Xác nhận mật khẩu mới

ĐỔI MẬT KHẨU

**Sao lưu & Khôi phục**

ⓘ Sao lưu dữ liệu của bạn để tránh mất mát thông tin quan trọng.

**Vùng nguy hiểm**

⚠ Xóa tài khoản sẽ xóa vĩnh viễn tất cả dữ liệu của bạn. Hành động này không thể hoàn tác.

- Cửa sổ Chat AI:

**Vissmart AI**  
Online

Xin chào! Tôi là trợ lý AI của Vissmart. Tôi có thể giúp bạn quản lý tài chính, phân tích chi tiêu và đưa ra lời khuyên về tiết kiệm. Bạn cần tôi giúp gì?

22:57

Làm sao để tiết kiệm?

23:18

Chào bạn! Vissmart AI rất vui được hỗ trợ bạn trong hành trình tiết kiệm này. Đây là một câu hỏi rất

💡 Gợi ý câu hỏi:

Chi tiêu tháng này? Làm sao để tiết kiệm? Xem ngân sách

Thu nhập của tôi? Phân tích chi tiêu Lời khuyên đầu tư

Nhập tin nhắn...

## VII. Hướng dẫn sử dụng hệ thống

### 1. Chuẩn bị môi trường

Để hệ thống hoạt động ổn định, máy tính cần cài đặt:

- **Java 17+** (để chạy Spring Boot Backend)
- **NodeJS 18+** (để chạy React Frontend)
- **MySQL 8.0** (để import dữ liệu)
- Các IDE như **VSCode / IntelliJ** (không bắt buộc, chỉ dùng để xem mã nguồn)

### 2. Khởi tạo cơ sở dữ liệu MySQL

Trong thư mục SQL/ của dự án có các file:

- **SQL.sql** – Tạo toàn bộ schema + bảng
- **TEST\_DATA.sql** – Dữ liệu mẫu
- **CLEAN\_TEST\_DATA.sql** – Xóa dữ liệu test nếu cần reset

Các bước thực hiện:

Bước 1: Mở MySQL Workbench

Bước 2: Tạo database: **CREATE DATABASE vissmart CHARACTER SET utf8mb4;**

Bước 3: Import file:

- Chạy **SQL.sql**
- Chạy **TEST\_DATA.sql** (nếu muốn demo sẵn dữ liệu)

### 3. Chạy Backend

Các bước thực hiện

Bước 1: Mở thư mục Backend trong IntelliJ IDEA/ VSCode

Bước 2: Kiểm tra file cấu hình kết nối database

- Mở file: **backend/src/main/resources/application.properties**
- Cập nhật các dòng:  
**spring.datasource.url=jdbc:mysql://localhost:3306/vissmart**  
**spring.datasource.username=root**  
**spring.datasource.password=your\_password**

Bước 3: Chuột phải vào file **backend/pom.xml** chọn **Add as Maven Project**

Bước 4: Sau khi Maven build xong, chạy file main:

- Mở file: **backend/src/main/java/com/fintrack/backend/BackendApplication**
- Bấm Run để khởi động backend.

### 4. Chạy Frontend

- Mở thư mục frontend và chạy:
  - o **npm install**
  - o **npm run dev**
- Frontend chạy tại: <http://localhost:3001>

### 5. Đăng kí/ đăng nhập và sử dụng

- Truy cập trang web, tạo tài khoản hoặc đăng nhập.
- Sau đó, người dùng có thể trải nghiệm các tính năng:
  - o Quản lý ví cá nhân
  - o Ghi nhận giao dịch thu chi
  - o Thiết lập và theo dõi ngân sách



- Xem báo cáo và biểu đồ
- Trò chuyện với AI
- Cài đặt thông tin người dùng và danh mục

## VIII. Đánh giá và kết luận

### 1. Ưu điểm

Trong quá trình xây dựng hệ thống Vissmart, nhóm đã đạt được kết quả tích cực sau:

Qua quá trình xây dựng hệ thống Vissmart, nhóm đã đạt được nhiều kết quả tích cực:

#### a) Kiến trúc rõ ràng – tách lớp đúng chuẩn OOP

- Backend triển khai theo mô hình Controller – Service – Repository – Entity giúp mã nguồn dễ bảo trì, mở rộng.
- Mỗi chức năng lớn được chia thành các module độc lập (Auth, Wallet, Transaction, Budget, Analytics...).

#### b) Áp dụng nhiều kỹ thuật hiện đại

- Spring Boot 3, Spring Security, JWT cho xác thực.
- React + Vite cho giao diện nhanh, nhẹ.
- MySQL quản lý dữ liệu rõ ràng bằng ERD.
- Tích hợp Gemini AI hỗ trợ gợi ý chi tiêu và trò chuyện.

#### c) Hoạt động ổn định, dữ liệu đồng bộ

- Khi thêm giao dịch → số dư ví cập nhật chính xác.
- Khi chi tiêu tăng → ngân sách tự động cộng dồn spent.
- Bộ lọc giao dịch, biểu đồ chi tiêu, cảnh báo ngân sách hoạt động đúng logic.

#### d) Giao diện thân thiện, dễ sử dụng

- Dashboard rõ ràng, hiển thị tổng quan tài chính.
- Các trang Wallet, Transactions, Budget... được thiết kế trực quan, phù hợp sinh viên Việt Nam.

#### e) Code tuân thủ nguyên tắc OOP

- Encapsulation: các entity đều private + getter/setter.
- Abstraction: service ẩn logic xử lý khỏi controller.
- Inheritance: repository kế thừa JpaRepository.
- Polymorphism: nhiều hành vi được định nghĩa thông qua interface và override.

### 2. Hạn chế

Mặc dù hệ thống hoàn thiện về mặt chức năng, vẫn còn một số hạn chế:

#### a) Chưa có hệ thống phân quyền nâng cao

- Mới chỉ có User cơ bản; chưa có vai trò Admin để quản lý toàn hệ thống.

#### b) Chưa triển khai đầy đủ các báo cáo nâng cao

- Báo cáo hiện tại chỉ ở mức cơ bản (theo tháng, theo danh mục).

#### c) AI mới ở mức hỗ trợ đơn giản

- Chưa phân tích sâu hành vi chi tiêu.
- Chưa cá nhân hóa gợi ý theo người dùng.

#### d) Giao diện chưa tối ưu cho mobile

- Các màn hình cần responsive tốt hơn để chạy trên điện thoại.

#### e) Chưa có cơ chế backup dữ liệu tự động

- Người dùng vẫn phải sao lưu thủ công hoặc nhờ DBMS.

### 3. Hướng phát triển

Trong tương lai, hệ thống Vissmart có thể tiếp tục mở rộng theo các hướng sau:

#### a) Hoàn thiện hệ thống phân quyền

- Thêm Admin Dashboard
- Thống kê người dùng, báo cáo tổng hợp

#### b) Mở rộng module AI

- Phân tích lịch sử chi tiêu dài hạn
- Dự đoán chi tiêu tương lai bằng ML
- Gợi ý ngân sách tối ưu hóa theo thói quen

#### c) Bổ sung báo cáo nâng cao

- Biểu đồ nhiều dạng hơn (đường, vùng, stacked...)
- Xuất báo cáo PDF, Excel
- Lịch sử thay đổi số dư ví

#### d) Xây dựng ứng dụng Mobile

- React Native hoặc Flutter
- Đồng bộ thời gian thực với backend

#### e) Đồng bộ dữ liệu với ngân hàng / e-wallet

- Mở rộng API kết nối thật các ví điện tử, ngân hàng
- Tự động import giao dịch

## IX. Phân công và mức độ đóng góp

### 1. Bảng phân chia công việc

| STT | Công việc                                               | Thành viên đảm nhận |
|-----|---------------------------------------------------------|---------------------|
| 1   | Thiết kế giao diện website                              | Ngọc Diệp           |
| 2   | Xây dựng Frontend                                       | Bảo Phúc            |
| 3   | Xây dựng Backend: Authentication & Authorization Module | Huy Hoàng           |
| 4   | Xây dựng Backend: Transaction Management Module         | Huy Hoàng           |
| 5   | Xây dựng Backend: Wallet & Account Management Module    | Ánh Dương           |
| 6   | Xây dựng Backend: Budget Management Module              | Ánh Dương           |
| 7   | Xây dựng Backend: Analytics & Reporting Module          | Ngọc Diệp           |
| 8   | Xây dựng Backend: AI Integration Module                 | Bảo Phúc            |
| 9   | Xây dựng Backend: Dashboard & Overview Module           | Bảo Phúc            |
| 10  | Xây dựng Backend: Configuration & Settings Module       | Ngọc Diệp           |

|    |                            |                       |
|----|----------------------------|-----------------------|
| 11 | Viết báo cáo               | Ánh Dương + Huy Hoàng |
| 12 | Review mã nguồn + sửa chữa | Cả nhóm               |

## 2. Tỷ lệ đóng góp từng thành viên

| STT | Họ tên             | Tỷ lệ đóng góp |
|-----|--------------------|----------------|
| 1   | Võ Văn Bảo Phúc    | 25%            |
| 2   | Trần Thị Ánh Dương | 25%            |
| 3   | Vũ Ngọc Diệp       | 25%            |
| 4   | Bùi Huy Hoàng      | 25%            |